

Teste Frontend React - Integração CEP e Clima

Prova técnica — Dev Frontend (React)

Objetivo: construir uma **aplicação React** que integra **APIs públicas** (CEP + clima), focando em **boas práticas, UX, testes e responsividade**.

Personas nas US: **Usuário, Integrador, Testador, Mantenedor** (Dev), **Avaliador**.

Escopo e Regras Gerais

- **Stacks:**
 - React 19+ com TypeScript
 - React Hook Form + Zod (validação)
 - TanStack Query (gerenciamento de estado servidor)
 - Vitest + Testing Library (testes)
 - React Router (navegação)
 - **Estilização (é permitido utilizar):**
 - Tailwind CSS
 - Styled-components
 - CSS Modules
 - Outras soluções CSS-in-JS ou metodologias CSS
 - **APIs Públicas:**
 - CEP primária: **BrasilAPI CEP v2** (<https://brasilapi.com.br/api/cep/v2/{cep}>)
 - CEP fallback: **ViaCEP** (<https://viacep.com.br/ws/{cep}/json/>)
 - Clima: **Open-Meteo Forecast** (<https://api.open-meteo.com/v1/forecast?...>)
 - Geocodificação: **Open-Meteo Geocoding** (<https://geocoding-api.open-meteo.com/v1/search?...>)
 - **Entrega:** Repositório com `/src`, `/tests`, `README.md`
 - **Design:** Layout responsivo, sem necessidade de Figma específico (criatividade do candidato)
-

US01 — Consulta de CEP com Interface Amigável

Como um Usuário, quero inserir um CEP em um formulário e visualizar o endereço completo de forma clara e organizada.

CA - Critérios de Aceitação:

- Formulário com campo de entrada para CEP (aceitar com/sem hífen)
- Validação de CEP em tempo real (8 dígitos)
- Exibir feedback visual durante carregamento
- Consultar **BrasilAPI CEP v2** como primária
- Em falha/indisponibilidade, fazer **fallback** para **ViaCEP**
- Mostrar dados do endereço retornados pela API quando disponível:
 - CEP formatado

- Logradouro
- Bairro
- Cidade
- UF
- Código IBGE
- Coordenadas
- Provedor utilizado (BrasilAPI/ViaCEP)
- Tratamento de erros com mensagens amigáveis
- Campo deve ser limpo após consulta bem-sucedida ou permitir nova consulta

NI - Notas de Implementação:

- Usar React Hook Form para gerenciamento do formulário
 - Aplicar máscara de CEP (XXXXX-XXX)
 - Debounce na digitação para evitar consultas excessivas
 - Estados de loading, sucesso e erro bem definidos
 - Mapear campos distintos dos provedores para um formato único
 - Implementar timeout e retry nas chamadas de API
-

US02 — Consulta de Clima Integrada ao CEP

Como um Usuário, quero consultar o clima atual e previsão do tempo baseado no CEP informado, com opção de escolher quantos dias de previsão.

CA:

- Seção/componente para exibir dados climáticos após consulta de CEP
- Selector para escolher número de dias de previsão (1-7 dias por exemplo)
- Reutilizar dados de CEP para obter coordenadas (lat/lon)
- Se não houver coordenadas, **geocodificar** por **cidade + uf** usando **Open-Meteo Geocoding**
- Consultar **Open-Meteo Forecast** para **clima atual e previsão diária**
- Exibir informações do clima atual:
 - Temperatura atual em Celsius
 - Sensação térmica
 - Umidade relativa
 - Data/hora da observação
- Exibir previsão diária:
 - Data
 - Temperatura mínima
 - Temperatura máxima
- Indicar localização (cidade/UF) e coordenadas
- Cache das consultas por (**lat, lon, days**) por **10 minutos**

NI:

- Usar TanStack Query para cache e gerenciamento de estado da API
- Componentes reutilizáveis para cartões de clima
- Formatação adequada de datas e temperaturas

- Ícones ou indicadores visuais para diferentes dados climáticos
 - Converter unidades do Open-Meteo para Celsius quando necessário
 - Implementar timeout nas chamadas de API climática
-

US03 — Interface Responsiva e Acessível

Como um Usuário, quero acessar a aplicação em diferentes dispositivos (mobile, tablet, desktop) com boa usabilidade.

CA:

- Layout responsivo que funciona em telas de 320px a 1920px+
- Navegação intuitiva entre seções
- Componentes com estados visuais claros (hover, focus, disabled)
- Feedback visual adequado para ações do usuário
- Contrast ratio adequado para textos
- Loading states em todas as operações assíncronas

NI:

- Usar CSS responsivo (Tailwind, styled-components, CSS Modules, etc.)
 - Componentes seguindo padrões de design system
 - Teste em diferentes tamanhos de tela
-

US04 — Tratamento de Erros e Estados de Loading

Como um Usuário, quero receber feedback claro sobre o status das operações e possíveis erros.

CA:

- Estados de loading com indicadores visuais apropriados
- Mensagens de erro amigáveis baseadas nos retornos da API:
 - CEP inválido
 - CEP não encontrado
 - Erro de conectividade
 - Timeout de requisição
- Opção de tentar novamente em caso de erro

NI:

- Componentes de Toast/Notification para feedback
 - Estados globais de erro bem estruturados
 - Retry automático ou manual conforme o tipo de erro
-

US05 — Histórico de Consultas

Como um Usuário, quero ver um histórico das minhas últimas consultas de CEP para reutilizar informações.

CA:

- Lista das últimas 5 ou 10 consultas realizadas
- Persistir histórico no localStorage
- Permitir clicar em item do histórico para recarregar dados
- Opção de limpar histórico
- Mostrar CEP, cidade/UF e timestamp da consulta

NI:

- Hook customizado para gerenciamento do histórico
 - Componente de lista reutilizável
 - Formatação de datas relativas ("há 2 minutos" por exemplo)
-

US06 — Testes Automatizados

Como um Mantenedor, quero testes automatizados cobrindo funcionalidades críticas para manter qualidade do código.

CA:

- **Testes unitários** para:
 - Validação de CEP
 - Formatação de dados
 - Hooks customizados
 - Utilitários/helpers
- **Testes de integração** para:
 - Fluxo completo de consulta CEP → exibição dados
 - Fluxo consulta clima após CEP
 - Tratamento de erros da API
 - Histórico de consultas
- **Testes de componente** para:
 - Renderização correta de dados
 - Interações do usuário
 - Estados de loading/error
- Cobertura focada em funcionalidades críticas

NI:

- Usar Vitest + Testing Library
 - Mocking da API para testes consistentes
 - Testes com diferentes cenários (sucesso, erro, loading)
-

US07 — Estrutura e Organização do Código

Como um Mantenedor, quero código bem estruturado e documentado para facilitar manutenção e evolução.

CA:

- Estrutura de pastas organizada seguindo boas práticas
- Componentes com responsabilidade única
- Hooks customizados para lógica reutilizável
- Types TypeScript bem definidos
- Separação clara entre componentes, services, utils e types

NI:

- ESLint e Prettier configurados
 - Import/export organizados
 - Nomenclatura consistente
-

US08 — Dockerização da Aplicação

Como um Operador/DevOps, quero executar a aplicação em um container Docker para facilitar o deployment e garantir consistência entre ambientes.

CA:

- Dockerfile funcional para build da aplicação
- Aplicação rodando corretamente em container
- Expor porta adequada para acesso externo
- Aplicação deve funcionar via container

NI:

- Usar imagem base adequada (nginx, node, etc.)
 - Documentar comandos Docker no README
 - Considerar .dockerignore para otimizar build
-

US09 — Documentação do Projeto (README.md)

Como um Avaliador/Mantenedor, quero documentação clara e completa para entender, executar e avaliar o projeto rapidamente.

CA:

- README.md completo e bem estruturado no repositório
- Instruções claras de instalação e execução
- Documentação das tecnologias e decisões técnicas
- Comandos para todas as operações (dev, build, test, docker)
- Estrutura do projeto explicada